# Fix plan

**Client:** Example Co

## Summary

Quick fixes the kit does: 1 · For a developer: 13 (≈5.4 h) · Only the client can do: 6

## 1. Urgent (critical)

### Exposed keys and passwords (4)

**Who:** Client · **Approx. time:** 1.3 h

| Key | Where | Where to revoke it |
|---|---|---|
| AWS access key | .env:1 | https://console.aws.amazon.com/iam/home#/security_credentials |
| Database URL with password | .env:2 | Database dashboard: change that user's password |
| Stripe live secret key | billing/stripe.js:1 | https://dashboard.stripe.com/apikeys |
| Supabase service_role key (full database access) | src/lib/supabase.ts:1 | https://supabase.com/dashboard/project/_/settings/api |

1. Go to this key's provider ("Where to revoke it" column) and **revoke** it (delete or disable). Until it is revoked it still works, even after removing it from the code.
2. Create a new key and store it in the server or hosting environment variables, or a secrets manager. Never in a project file.
3. Change the code to read it from the environment variable (e.g. `process.env.NAME` or `os.environ["NAME"]`).
4. If it was in git history: purge it with `git filter-repo` or ask the developer. If the repository was ever public, treat it as compromised even after revoking.
5. Check the provider for unusual use of the key (charges, logins) since it was committed.

### Firebase rules allow anyone access (if true)

**Where:** `firestore.rules:4` · **Who:** Developer · **Approx. time:** 45 min

1. Replace the rules with the kit's `firebase-rules-example.txt`, adapted to your collections.
2. Publish the rules and test while logged out.

## 2. Immediately (high)

### Exposed keys and passwords (2)

**Who:** Client · **Approx. time:** 40 min

| Key | Where | Where to revoke it |
|---|---|---|
| Anthropic API key | .mcp.json:10 | https://console.anthropic.com/settings/keys |
| OpenAI API key | config.py (commit de94084f) | https://platform.openai.com/api-keys |

1. Go to this key's provider ("Where to revoke it" column) and **revoke** it (delete or disable). Until it is revoked it still works, even after removing it from the code.

2. Create a new key and store it in the server or hosting environment variables, or a secrets manager. Never in a project file.
3. Change the code to read it from the environment variable (e.g. `process.env.NAME` or `os.environ["NAME"]`).
4. If it was in git history: purge it with `git filter-repo` or ask the developer. If the repository was ever public, treat it as compromised even after revoking.
5. Check the provider for unusual use of the key (charges, logins) since it was committed.

## Claude Code set to skip all permission prompts

**Where:** `.claude/settings.json` · **Who:** Developer · **Approx. time:** 10 min

1. In `.claude/settings.json`, remove `"defaultMode": "bypassPermissions"`.
2. Allow only specific commands, such as `Bash(npm test)`.

## Hidden characters in AI instructions file (3)

**Where:** `CLAUDE.md:2` · **Who:** Developer · **Approx. time:** 20 min

1. Open the file, delete the invisible characters and review what instructions they hid.
2. Check git history for who added them.

## SQL built by joining text (possible injection)

**Where:** `api/app.py:3` · **Who:** Developer · **Approx. time:** 30 min

1. Change the query to pass values as parameters (`$1`, `?`, `%s`) instead of joining text.

## Secret exposed to the browser through NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY

**Where:** `src/lib/supabase.ts:2` · **Who:** Developer · **Approx. time:** 1 h

1. Remove the public prefix (`NEXT_PUBLIC_`/`VITE_`) from that variable.
2. Move the use of the secret to a server route or function.
3. Rotate the key, because it was already published in the site.

## Table 'customers' has no row-level security

**Where:** `supabase/migrations/001_init.sql:1` · **Who:** Developer · **Approx. time:** 45 min

1. Review and run the kit's `supabase-rls.sql` in the Supabase SQL editor (turns on row-level security).
2. Adjust the policies so each user sees only their own rows.
3. Test the app while logged out: it must not read any data.

## Database policy allows everyone (using true)

**Where:** `supabase/migrations/001_init.sql:4` · **Who:** Developer · **Approx. time:** 45 min

1. Replace `using (true)` with an owner condition, e.g. `using (auth.uid() = user_id)`.
2. Test logged out and as a different user.

## Environment file committed to the repository

**Where:** `.env` · **Who:** The kit does it · **Approx. time:** 10 min

1. The kit adds `.env` to `.gitignore` and creates `.env.example` with variable names only (no values).
2. Remove it from the repository but keep it on disk: `git rm --cached .env`, then commit.
3. Rotate the values it held (see the keys step), because they remain in history.

## 3. This week (medium)

### Claude Code pre-approved to run broad shell commands

**Where:** `.claude/settings.json` · **Who:** Developer · **Approx. time:** 10 min

1. In `.claude/settings.json`, replace rules like `Bash(*)` with the exact commands the project uses.

### All project MCP servers are trusted automatically

**Where:** `.claude/settings.json` · **Who:** Developer · **Approx. time:** 5 min

1. Remove `enableAllProjectMcpServers` and approve MCP servers by name.

### TLS certificate checks turned off

**Where:** `api/app.py:4` · **Who:** Developer · **Approx. time:** 30 min

1. Download the CA certificate from the database or service provider.
2. Configure the connection to use it and verify, e.g. in Node `ssl: { ca: fs.readFileSync('ca.pem'), rejectUnauthorized: true }`.
3. Test the connection.

### Debug mode enabled

**Where:** `api/app.py:5` · **Who:** Developer · **Approx. time:** 10 min

1. Read debug mode from an environment variable and keep it off in production.

## 4. This month (low)

### MCP server 'crm' runs an unpinned package

**Where:** `.mcp.json` · **Who:** Developer · **Approx. time:** 10 min

1. Pin the MCP server package to an exact version (`package@1.2.3`).

### No lockfile for JavaScript dependencies

**Where:** `package.json` · **Who:** Developer · **Approx. time:** 5 min

1. Run `npm install` (or `pnpm install` / `yarn`) and commit the lockfile it creates.

## 5. Ready-made files in this kit

Each folder holds the files for that project. **Apply quick fixes** copies the AI rules (`AGENTS.md`, `CLAUDE.md`), `.env.example` and the key blocker (`.pre-commit-config.yaml`) into the project and appends lines to `.gitignore`. It never overwrites files or changes code. The auditor sets up the GitHub workflow and Dependabot, because they change how the repository behaves.

- **tienda-en-linea** (`tienda-en-linea/`): `.env.example`, `.github/dependabot.yml`, `.github/workflows/guardmycode.yml`, `.gitignore-additions.txt`, `.pre-commit-config.yaml`, `AGENTS.md`, `CLAUDE.md`, `firebase-rules-example.txt`, `fix-with-claude.md`, `supabase-rls.sql`

## 6. Lasting protection (monthly plan)

- **Block keys before every commit:** `.pre-commit-config.yaml` with Gitleaks. Each developer runs `pip install pre-commit` and `pre-commit install` once.

- **Check every change:** `.github/workflows/guardmycode.yml` runs this scanner on every pull request and blocks high-risk problems.
- **Vulnerable libraries:** `.github/dependabot.yml` proposes security updates automatically.
- **Rules for the AI:** `AGENTS.md` (and `CLAUDE.md`) tell Claude, Cursor, Copilot and others what they must never do.
- **On GitHub:** Settings → Code security → turn on *Secret scanning* and *Push protection* where the plan allows, and require review before merging to main.
- **Monthly re-scan** with GuardMyCode and a one-page report for the owner.

## 7. For the developer: fix with Claude Code

Open the project in Claude Code (or Cursor) and paste the contents of `fix-with-claude.md` from the project's folder. Review each change before it goes live.